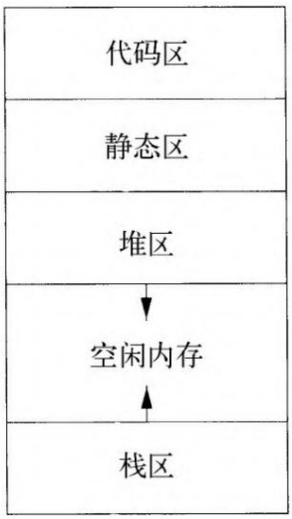
- 运行时刻内存
- 活动树
- 计算题例子

- 运行时刻内存

运行时刻内存是为一个目标程序运行时（即进程）而设计的逻辑地址空间的划分。

一个目标程序在逻辑地址空间的运行时刻映像包含数据区和代码区，如图7-1所示。某个语言（比如C++）在某个操作系统（比如Linux）上的编译器可能按照这种方式划分存储空间。

图7-1 运行时刻内存被划分成代码区和数据区的典型方式




生成的目标代码的大小在编译时刻就已经固定下来了，因此编译器可以将可执行目标代码放在一个静态确定的区域：代码区。这个区通常位于存储的低端。类似地，程序的某些数据对象的大小可以在编译时刻知道，它们可以被放置在另一个称为静态区的区域中，该区域可以被静态确定。放置在这个区域的数据对象包括全局常量和编译器产生的数据，比如用于支持垃圾回收的信息等。之所以要将尽可能多的数据对象进行静态分配，是因为这些对象的地址可以被编译到目标代码中。在Fortran的早期版本中，所有数据对象都可以进行静态分配。

为了将运行时刻的空间利用率最大化，另外两个区域——栈和堆被放在剩余地址空间的相对两端。这些区域是动态的，它们的大小会随着程序运行而改变。这两个区域根据需要向对方增长。栈区用来存放称为活动记录的数据结构，这些活动记录在函数调用过程中生成。

在实践中，栈向较低地址方向增长，而堆向较高地址方向增长。然而，在本章及下一章中，我们将假定栈向较高地址方向增长，以便我们能够在所有例子中方便地使用正的偏移量。

问题。这些问题需要谨慎对待，因为程序文本中的同一个名字可能在运行时刻指向不同的存储位置。两个形容词静态（static）和动态（dynamic）分别表示编译时刻和运行时刻。如果编译器只需要通过观察程序文本即可做出某个存储分配决定，而不需要观察该程序在运行时做了什么，我们就认为这个存储分配决定是静态的。反过来，如果只有在程序运行时才能做出决定，那么这个决定就是动态的。很多编译器使用下列两种策略的某种组合进行动态存储分配：

1）*栈式存储*。一个过程的局部名字在栈中分配空间。我们将从7.2节开始讨论“运行时刻栈”。这种栈支持通常的过程调用/返回策略。

2）*堆存储*。有些数据的生命周期要比创造它的某次过程调用更长，这些数据通常被分配在一个可复用存储的“堆”中。我们将从7.4节开始讨论堆管理。堆是虚拟内存的一个区域，它允许对象或其他数据元素在被创建时获得存储空间，并在数据变得无效时释放该存储空间。

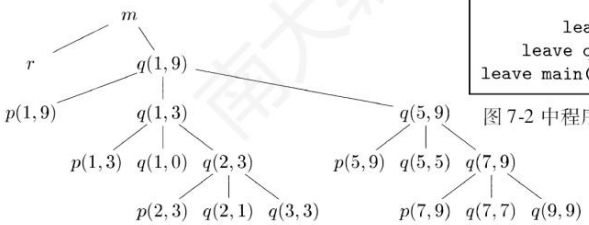
- 活动树（activation tree）

形容一个进程发生的所有过程调用次序和调用关系的结构称为活动树。

- 程序运行的所有过程活动可以用树表示
  - 每个结点对应于一个过程活动
  - 根结点对应于main过程的活动
  - 过程$p$的某次活动对应的结点的所有子结点
    - 表示此次活动所调用的各个过程活动
    - 从左向右，表示调用的先后顺序
  - 又称为调用树 (Call Tree)

## 活动树的例子

- 快速排序程序
  - 过程调用 (返回) 序列和活动树的前序 (后序) 遍历对应
  - 假定当前活动对应结点$N$，那么所有尚未结束的活动对应于$N$及其祖先结点




```
enter main()
    enter readArray()
    leave readArray()
    enter quicksort(1,9)
        enter partition(1,9)
        leave partition(1,9)
        enter quicksort(1,3)
            ...
        leave quicksort(1,3)
        enter quicksort(5,9)
            ...
        leave quicksort(5,9)
    leave quicksort(1,9)
leave main()
```

图 7-2 中程序的可能的活动序列

7

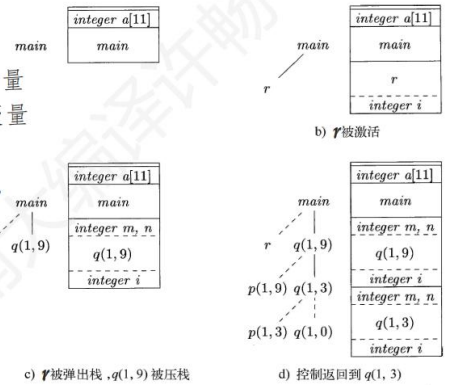
使用控制栈来管理进程里的过程调用（即函数调用）。一个程序发生（嵌套的）过程调用时，每个过程调用会在控制栈中用一个活动记录来管理。
一个活动记录（record）对应着树中的一个节点。一个运行时刻栈对应着树中的从根节点往下的一条路径中的节点排列。




图7-5 一个概括性的活动记录

## 运行时刻栈的例子

- 说明
  - $a$[11]为全局变量
  - main无局部变量
  - $r$有局部变量$i$
  - $q$有局部变量$i$，和参数$m, n$




c) $r$被弹出栈，$q(1,9)$被压栈　　b) $r$被激活　　d) 控制返回到 $q(1,3)$

1）临时值。比如当表达式求值过程中产生的中间结果无法存放在寄存器中时，就会生成这些临时值。

2）对应于这个活动记录的过程的局部数据。

3）保存的机器状态，其中包括对此过程的此次调用之前的机器状态信息。这些信息通常包括返回地址（程序计数器的值，被调用过程必须返回到该值所指位置）和一些寄存器中的内容（调用过程会使用这些内容，被调用过程必须在返回时恢复这些内容）。

4）一个“访问链”。当被调用过程需要其他地方（比如另一个活动记录）的某个数据时需要使用访问链进行定位。访问链将在7.3.5节中讨论。

5）一个控制链（control link），指向调用者的活动记录。

6）当被调用函数有返回值时，要有一个用于存放这个返回值的空间。不是所有的被调用过程都有返回值，即使有，我们也可能倾向于将该值放到一个寄存器中以提高效率。

7）调用过程使用的实在参数（actual parameter）。这些值通常将尽

可能地放在寄存器中，而不是放在活动记录中，因为放在寄存器中会得到更好的效率。然而，我们仍然为它们预留了相应的空间，使得我们的活动记录具有完全的通用性。

**来细节看下 caller 和 callee 的活动记录：**

活动记录内部设计的目的是为了满足如下要求

## 调用/返回代码序列的要求

- 数据方面
  - 能够把参数正确地传递给被调用者
  - 能够把返回值传递给调用者
- 控制方面
  - 能够正确转到被调用过程的代码开始位置
  - 能够正确转回调用者的调用位置 (的下一条指令)
- 调用代码序列与活动记录的布局相关

# 调用代码序列的例子

- Calling sequence
  - 调用者计算实在参数的值
  - 将返回地址和原*top_sp*存放到被调用者的活动记录中；调用者增加*top_sp*的值 (越过了调用者的局部数据和临时变量、以及被调用者的参数和机器状态字段)
  - 被调用者保存寄存器值和其他状态字段
  - 被调用者初始化局部数据，开始运行
- Return sequence
  - 被调用者将返回值放到与参数相邻的位置
  - 恢复*top_sp*和寄存器，跳转到返回地址

# 调用者/被调用者的活动记录

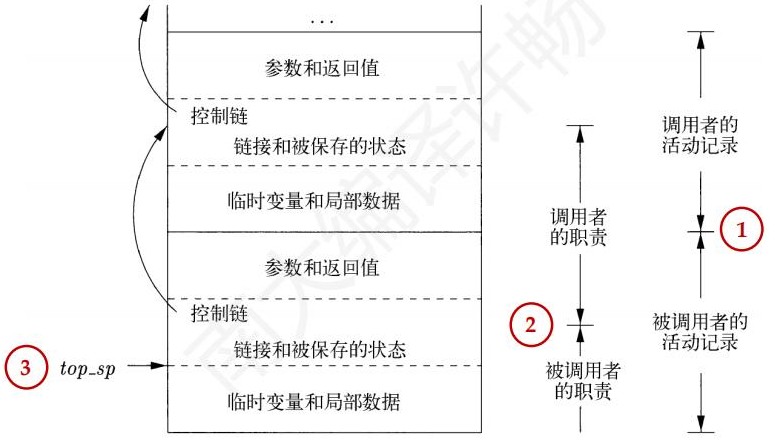



图 7-7　调用者和被调用者之间的任务划分

- **计算题例子**

**CP6 p30（这个 ppt 上有一定错误）**